

## **EXPERIMENT – 4**

**AIM:** To perform and analyze SQL queries using INNER JOIN, LEFT JOIN, SELF JOIN, 3-way JOIN, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT operations on the Employee–Department–Project database, and to compare their execution plans using EXPLAIN.

### **SOURCE CODE:**

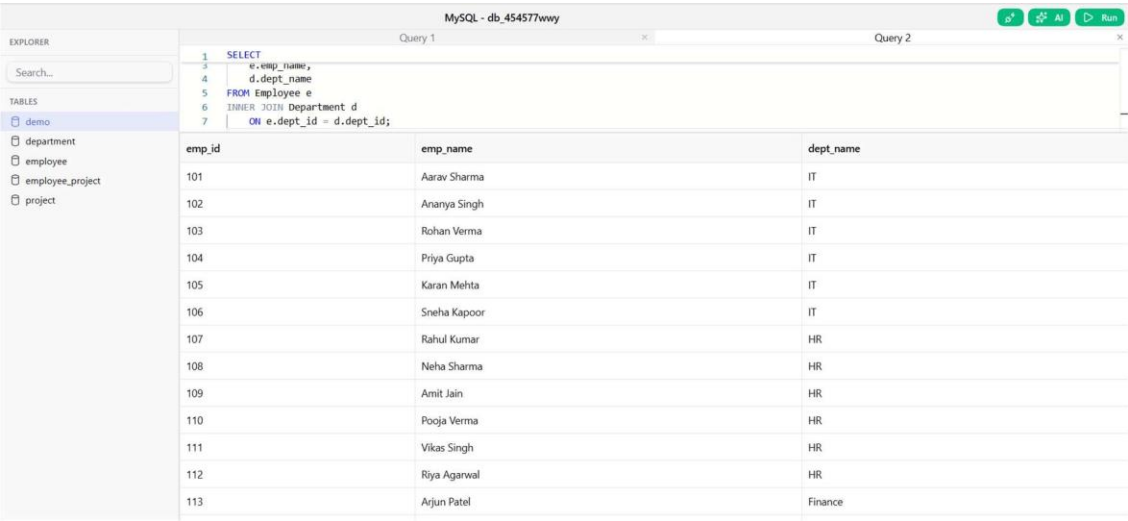
#### **USING EXPERIMENT – 3 DATABASE**

### **SQL QUERIES:**

#### **INNER JOIN**

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id;
```

#### **OUTPUT:**



The screenshot shows a MySQL IDE window titled 'MySQL - db\_454577wwy'. On the left is an 'EXPLORER' pane with a search bar and a 'TABLES' list containing 'demo', 'department', 'employee', 'employee\_project', and 'project'. The 'demo' table is selected. The main editor shows 'Query 1' with the following SQL code:

```
1 SELECT
2     e.emp_name,
3     d.dept_name
4 FROM Employee e
5 INNER JOIN Department d
6     ON e.dept_id = d.dept_id;
```

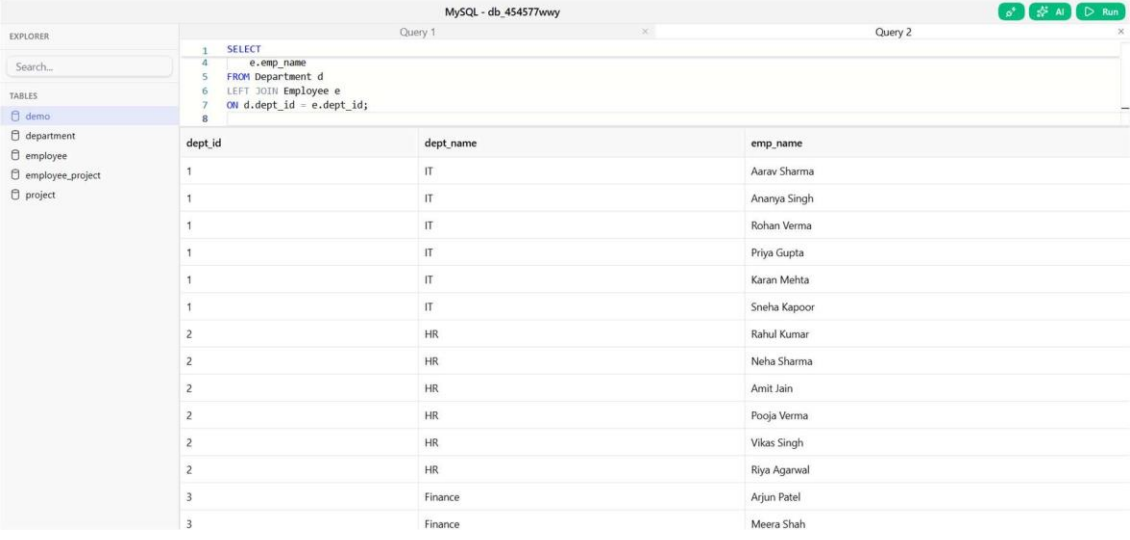
Below the query editor, the results of the query are displayed in a table with three columns: 'emp\_id', 'emp\_name', and 'dept\_name'. The results show 13 rows of data.

| emp_id | emp_name     | dept_name |
|--------|--------------|-----------|
| 101    | Aarav Sharma | IT        |
| 102    | Ananya Singh | IT        |
| 103    | Rohan Verma  | IT        |
| 104    | Priya Gupta  | IT        |
| 105    | Karan Mehta  | IT        |
| 106    | Sneha Kapoor | IT        |
| 107    | Rahul Kumar  | HR        |
| 108    | Neha Sharma  | HR        |
| 109    | Amit Jain    | HR        |
| 110    | Pooja Verma  | HR        |
| 111    | Vikas Singh  | HR        |
| 112    | Riya Agarwal | HR        |
| 113    | Arjun Patel  | Finance   |

## LEFT JOIN

```
SELECT
    d.dept_id,
    d.dept_name,
    e.emp_name
FROM Department d
LEFT JOIN Employee e
    ON d.dept_id = e.dept_id;
```

## OUTPUT:



The screenshot shows a MySQL IDE window titled 'MySQL - db\_454577wwy'. On the left is an 'EXPLORER' pane with a search bar and a 'TABLES' list containing 'demo', 'department', 'employee', 'employee\_project', and 'project'. The main area displays 'Query 1' with the following SQL code:

```
1 SELECT
4     e.emp_name
5 FROM Department d
6 LEFT JOIN Employee e
7     ON d.dept_id = e.dept_id;
8
```

Below the query, the results are shown in a table with three columns: 'dept\_id', 'dept\_name', and 'emp\_name'. The results list 15 employees, grouped by department: 5 employees in IT (dept\_id 1), 6 employees in HR (dept\_id 2), and 4 employees in Finance (dept\_id 3).

| dept_id | dept_name | emp_name     |
|---------|-----------|--------------|
| 1       | IT        | Aarav Sharma |
| 1       | IT        | Ananya Singh |
| 1       | IT        | Rohan Verma  |
| 1       | IT        | Priya Gupta  |
| 1       | IT        | Karan Mehta  |
| 1       | IT        | Sneha Kapoor |
| 2       | HR        | Rahul Kumar  |
| 2       | HR        | Neha Sharma  |
| 2       | HR        | Amit Jain    |
| 2       | HR        | Pooja Verma  |
| 2       | HR        | Vikas Singh  |
| 2       | HR        | Riya Agarwal |
| 3       | Finance   | Arjun Patel  |
| 3       | Finance   | Meera Shah   |

## SELF JOIN

```
SELECT
    e1.emp_name AS Employee1,
    e2.emp_name AS Employee2,
    e1.dept_id
FROM Employee e1
JOIN Employee e2
    ON e1.dept_id = e2.dept_id
    AND e1.emp_id < e2.emp_id;
```

OUTPUT:

MySQL - db\_454577wwy

EXPLORER

Search...

TABLES

demo

department

employee

employee\_project

project

Query 1

Query 2

1 SELECT

4 e1.dept\_id

5 FROM Employee e1

6 JOIN Employee e2

7 ON e1.dept\_id = e2.dept\_id

8 AND e1.emp\_id < e2.emp\_id;

| Employee1    | Employee2    | dept_id |
|--------------|--------------|---------|
| Aarav Sharma | Ananya Singh | 1       |
| Aarav Sharma | Rohan Verma  | 1       |
| Aarav Sharma | Priya Gupta  | 1       |
| Aarav Sharma | Karan Mehta  | 1       |
| Aarav Sharma | Sneha Kapoor | 1       |
| Ananya Singh | Rohan Verma  | 1       |
| Ananya Singh | Priya Gupta  | 1       |
| Ananya Singh | Karan Mehta  | 1       |
| Ananya Singh | Sneha Kapoor | 1       |
| Rohan Verma  | Priya Gupta  | 1       |
| Rohan Verma  | Karan Mehta  | 1       |
| Rohan Verma  | Sneha Kapoor | 1       |
| Priya Gupta  | Karan Mehta  | 1       |
| Priya Gupta  | Sneha Kapoor | 1       |

3-WAY JOIN

```
SELECT
    e.emp_name,
    d.dept_name,
    p.project_name
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
JOIN Employee_Project ep
    ON e.emp_id = ep.emp_id
JOIN Project p
    ON ep.project_id = p.project_id;
```

OUTPUT:

MySQL - db\_454577wwy

EXPLORER

Search...

TABLES

demo

department

employee

employee\_project

project

Query 1

Query 2

1 SELECT

4 e.emp\_name

5 FROM Employee e

6 JOIN Department d

7 ON e.dept\_id = d.dept\_id

8 JOIN Employee\_Project ep

9 ON e.emp\_id = ep.emp\_id

10 JOIN Project p

11 ON ep.project\_id = p.project\_id;

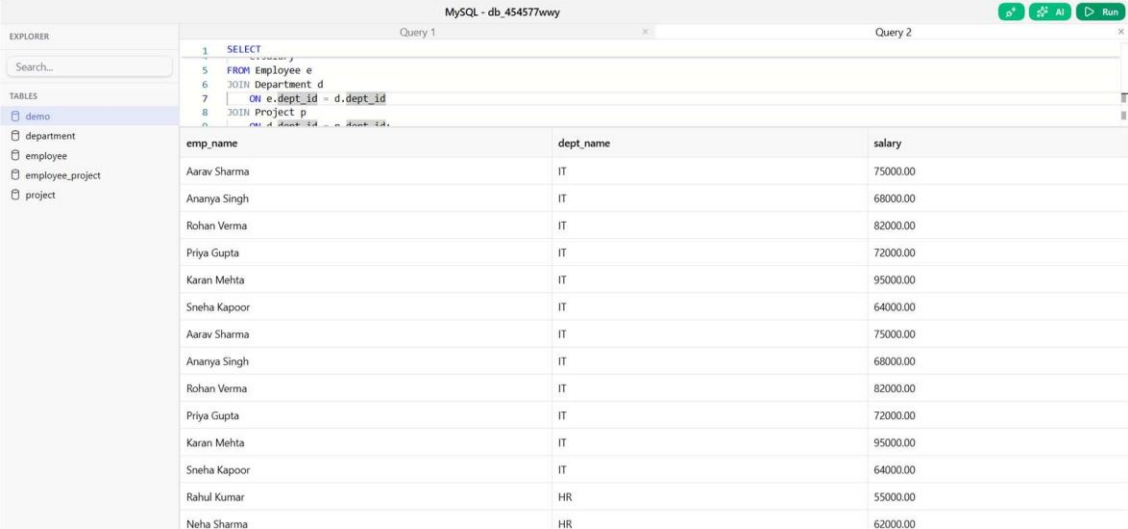
| emp_name     | dept_name | project_name       |
|--------------|-----------|--------------------|
| Aarav Sharma | IT        | AI Chatbot         |
| Ananya Singh | IT        | AI Chatbot         |
| Karan Mehta  | IT        | AI Chatbot         |
| Rohan Verma  | IT        | Cloud Migration    |
| Priya Gupta  | IT        | Cloud Migration    |
| Sneha Kapoor | IT        | Cloud Migration    |
| Rahul Kumar  | HR        | Employee Training  |
| Neha Sharma  | HR        | Employee Training  |
| Amit Jain    | HR        | Employee Training  |
| Pooja Verma  | HR        | Employee Training  |
| Vikas Singh  | HR        | Employee Training  |
| Riya Aganwal | HR        | Employee Training  |
| Aritam Patel | Finance   | Financial Analysis |

```

SELECT
    e.emp_name,
    d.dept_name,
    e.salary
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
JOIN Project p
    ON d.dept_id = p.dept_id;

```

## OUTPUT:



| emp_name     | dept_name | salary   |
|--------------|-----------|----------|
| Aarav Sharma | IT        | 75000.00 |
| Ananya Singh | IT        | 68000.00 |
| Rohan Verma  | IT        | 82000.00 |
| Priya Gupta  | IT        | 72000.00 |
| Karan Mehta  | IT        | 95000.00 |
| Sneha Kapoor | IT        | 64000.00 |
| Aarav Sharma | IT        | 75000.00 |
| Ananya Singh | IT        | 68000.00 |
| Rohan Verma  | IT        | 82000.00 |
| Priya Gupta  | IT        | 72000.00 |
| Karan Mehta  | IT        | 95000.00 |
| Sneha Kapoor | IT        | 64000.00 |
| Rahul Kumar  | HR        | 55000.00 |
| Neha Sharma  | HR        | 62000.00 |

## CORRELATED SUBQUERY

```

SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    e.dept_id
FROM Employee e
WHERE e.salary >
(
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
);

```

OUTPUT:

MySQL - db\_454577wwy

Query 1

Query 2

1 SELECT  
2 FROM Employee e  
3 WHERE e.salary >  
4 (  
5 SELECT AVG(e2.salary)  
6 FROM Employee e2  
7 )

| emp_id | emp_name      | salary   | dept_id |
|--------|---------------|----------|---------|
| 103    | Rohan Verma   | 82000.00 | 1       |
| 105    | Karan Mehta   | 95000.00 | 1       |
| 110    | Pooja Verma   | 67000.00 | 2       |
| 112    | Riya Agarwal  | 71000.00 | 2       |
| 113    | Arjun Patel   | 88000.00 | 3       |
| 114    | Meera Shah    | 92000.00 | 3       |
| 116    | Kavya Iyer    | 85000.00 | 3       |
| 118    | Nisha Reddy   | 90000.00 | 3       |
| 121    | Mohit Bansal  | 74000.00 | 4       |
| 122    | Isha Malhotra | 81000.00 | 4       |
| 124    | Tanya Arora   | 77000.00 | 4       |
| 125    | Dev Malhotra  | 98000.00 | 5       |
| 126    | Aditi Mishra  | 91000.00 | 5       |
| 128    | Shreya Das    | 93000.00 | 5       |

EXISTS

```
SELECT
    e.emp_id,
    e.emp_name
FROM Employee e
WHERE EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
);
```

OUTPUT:

MySQL - db\_454577wwy

Query 1

Query 2

4 FROM Employee e  
5 WHERE EXISTS  
6 (  
7 SELECT 1  
8 FROM Employee\_Project ep  
9 WHERE ep.emp\_id = e.emp\_id  
10 )

| emp_id | emp_name     |
|--------|--------------|
| 101    | Aarav Sharma |
| 102    | Ananya Singh |
| 103    | Rohan Verma  |
| 104    | Priya Gupta  |
| 105    | Karan Mehta  |
| 106    | Sneha Kapoor |
| 107    | Rahul Kumar  |
| 108    | Neha Sharma  |
| 109    | Amit Jain    |
| 110    | Pooja Verma  |
| 111    | Vikas Singh  |
| 112    | Riya Agarwal |
| 113    | Arjun Patel  |
| 114    | Meera Shah   |

## SIMULATED INTERSECT

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE e.dept_id = 1

AND EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
        AND ep.project_id = 201
);
```

## OUTPUT:



The screenshot shows the MySQL IDE interface. The left sidebar displays the 'EXPLORER' with a search bar and a list of tables: 'demo', 'department', 'employee', 'employee\_project', and 'project'. The main window shows the SQL query for 'Query 2' and its results. The query is a simulated intersect query. The results table has two columns: 'emp\_id' and 'emp\_name'.

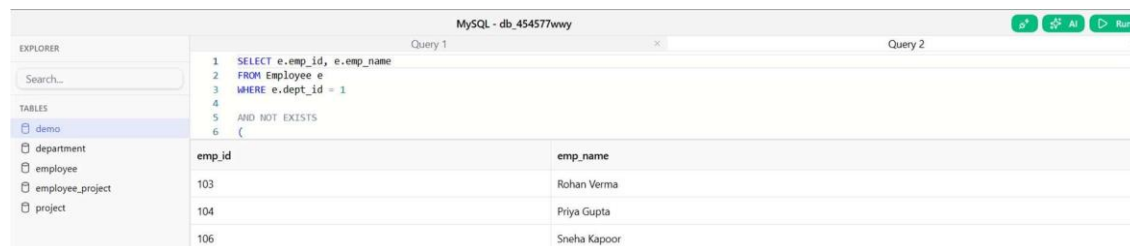
| emp_id | emp_name     |
|--------|--------------|
| 101    | Aarav Sharma |
| 102    | Ananya Singh |
| 105    | Karan Mehta  |

## SIMULATED EXCEPT

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE e.dept_id = 1

AND NOT EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
        AND ep.project_id = 201
);
```

## OUTPUT:



The screenshot shows the MySQL IDE interface. The left sidebar displays the 'EXPLORER' with a search bar and a list of tables: 'demo', 'department', 'employee', 'employee\_project', and 'project'. The main window shows the SQL query for 'Query 2' and its results. The query is a simulated except query. The results table has two columns: 'emp\_id' and 'emp\_name'.

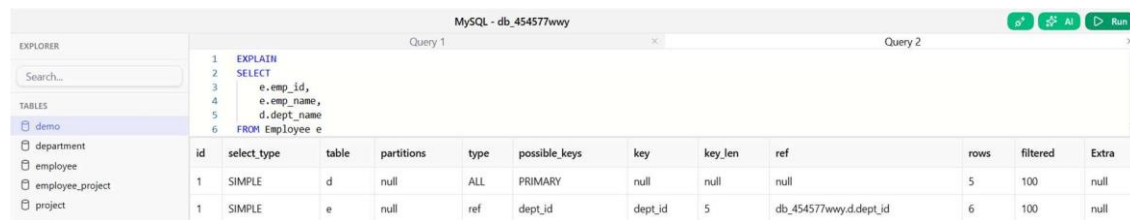
| emp_id | emp_name     |
|--------|--------------|
| 103    | Rohan Verma  |
| 104    | Priya Gupta  |
| 106    | Sneha Kapoor |

## COMPARING EXECUTION PLANS USING EXPLAIN

### EXPLAIN – INNER JOIN

```
EXPLAIN
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id;
```

#### OUTPUT:



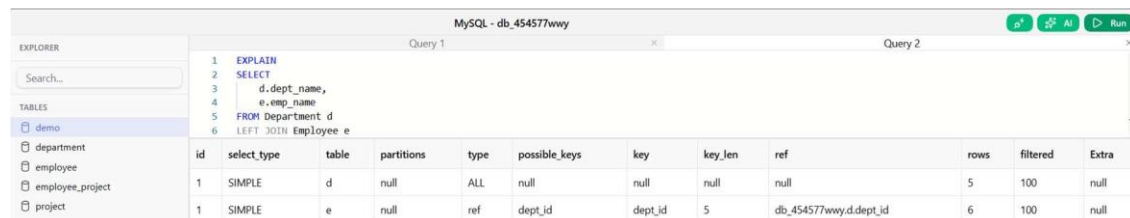
The screenshot shows the MySQL IDE interface with a query editor and an EXPLAIN output table. The query is an INNER JOIN between Employee and Department tables. The EXPLAIN output shows two rows: one for the Department table (id 1, select\_type SIMPLE, type ALL) and one for the Employee table (id 1, select\_type SIMPLE, type ref, key dept\_id). The rows column indicates 5 rows for Department and 6 rows for Employee.

| id | select_type | table | partitions | type | possible_keys | key     | key_len | ref                    | rows | filtered | Extra |
|----|-------------|-------|------------|------|---------------|---------|---------|------------------------|------|----------|-------|
| 1  | SIMPLE      | d     | null       | ALL  | PRIMARY       | null    | null    | null                   | 5    | 100      | null  |
| 1  | SIMPLE      | e     | null       | ref  | dept_id       | dept_id | 5       | db_454577www.d.dept_id | 6    | 100      | null  |

### EXPLAIN – LEFT JOIN

```
EXPLAIN
SELECT
    d.dept_name,
    e.emp_name
FROM Department d
LEFT JOIN Employee e
    ON d.dept_id = e.dept_id;
```

#### OUTPUT:



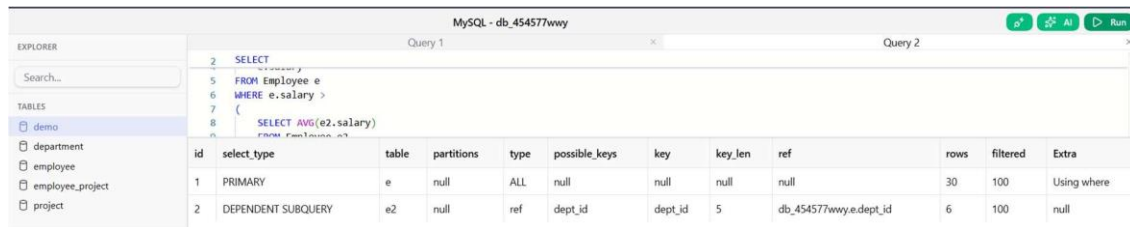
The screenshot shows the MySQL IDE interface with a query editor and an EXPLAIN output table. The query is a LEFT JOIN between Department and Employee tables. The EXPLAIN output shows two rows: one for the Department table (id 1, select\_type SIMPLE, type ALL) and one for the Employee table (id 1, select\_type SIMPLE, type ref, key dept\_id). The rows column indicates 5 rows for Department and 6 rows for Employee.

| id | select_type | table | partitions | type | possible_keys | key     | key_len | ref                    | rows | filtered | Extra |
|----|-------------|-------|------------|------|---------------|---------|---------|------------------------|------|----------|-------|
| 1  | SIMPLE      | d     | null       | ALL  | null          | null    | null    | null                   | 5    | 100      | null  |
| 1  | SIMPLE      | e     | null       | ref  | dept_id       | dept_id | 5       | db_454577www.d.dept_id | 6    | 100      | null  |

## EXPLAIN – CORRELATED SUBQUERY

```
EXPLAIN
SELECT
    e.emp_name,
    e.salary
FROM Employee e
WHERE e.salary >
(
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
);
```

### OUTPUT:

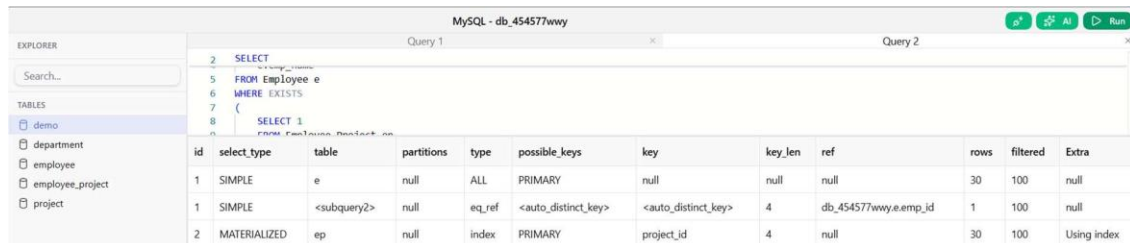


| id | select_type        | table | partitions | type | possible_keys | key     | key_len | ref                    | rows | filtered | Extra       |
|----|--------------------|-------|------------|------|---------------|---------|---------|------------------------|------|----------|-------------|
| 1  | PRIMARY            | e     | null       | ALL  | null          | null    | null    | null                   | 30   | 100      | Using where |
| 2  | DEPENDENT SUBQUERY | e2    | null       | ref  | dept_id       | dept_id | 5       | db_454577www.e.dept_id | 6    | 100      | null        |

## EXPLAIN – EXISTS

```
EXPLAIN
SELECT
    e.emp_id,
    e.emp_name
FROM Employee e
WHERE EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
);
```

### OUTPUT:



| id | select_type  | table       | partitions | type   | possible_keys       | key                 | key_len | ref                   | rows | filtered | Extra       |
|----|--------------|-------------|------------|--------|---------------------|---------------------|---------|-----------------------|------|----------|-------------|
| 1  | SIMPLE       | e           | null       | ALL    | PRIMARY             | null                | null    | null                  | 30   | 100      | null        |
| 1  | SIMPLE       | <subquery2> | null       | eq_ref | <auto_distinct_key> | <auto_distinct_key> | 4       | db_454577www.e.emp_id | 1    | 100      | null        |
| 2  | MATERIALIZED | ep          | null       | index  | PRIMARY             | project_id          | 4       | null                  | 30   | 100      | Using index |